

Autoresearch Loops

A Causal Study of the Instruction File

Christian Block

September 19, 2026

Research project with Prof. Ramesh

Nobody knows why these loops work

- March 2026: autoresearch ships
- Forked several thousand times since
- An agent runs its own experiments
- Nobody can say *which instruction* worked

Visual: full-bleed screenshot of `github.com/karpathy/autoresearch`, star and fork counts visible

The architecture is three files



- No scheduler. No driver program.
- The cycle exists only as English prose
- `program.md` is not configuration — it is the controller

It already works in production. That is the problem.

- Shopify Liquid, PR #2056
- 53% faster parse and render
- ~120 experiments, 93 commits
- Author: “probably overfitted”
- Never merged

Visual: screenshot of the PR header with the OPEN badge

Two questions

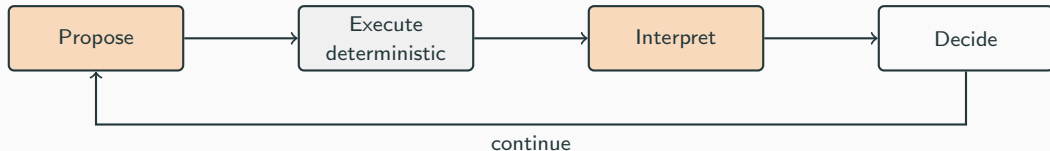
RQ1

Which components of the instruction file **causally determine** which aspects of the loop's behaviour, and how large is each effect?

RQ2

Do the components act **independently** of one another?

The loop under study



- Shaded = model call Unshaded = harness
- Search space: hyperparameters, not code
- The model never computes a number

Stop reading the prompt. Start configuring it.

- One template, five variable slots
- Everything else byte-identical
- Each slot written at two levels
- A file becomes a vector

$$c = (c_1, \dots, c_5), \quad c_i \in \{0, 1\}$$

$$c = (1, 0, 1, 0, 1)$$

$2^5 = 32$ distinct instruction files

The five components

	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	undefined	5-fold CV accuracy
<i>B</i>	Search breadth	one family	both families
<i>S</i>	Stopping rule	fixed 3	adaptive, up to 8
<i>O</i>	Reporting style	terse	full sentences
<i>E</i>	Search emphasis	explore first	exploit first

- Categories taken from a real `program.md`
- All ten paragraphs LLM-written, then frozen

Run the whole space

$$N = \underbrace{2^5}_{32 \text{ configs}} \times \underbrace{D}_{2 \text{ tasks}} \times \underbrace{R}_{20 \text{ reps}} = 1,280 \text{ runs}$$

- Constructed, not observed
- Confounding gone **by design**, not adjustment
- Every interaction estimable from every main effect
- A fraction would have to **assume** RQ2's answer

What I held fixed — and one thing I did not

Held constant

- Pinned model snapshot
- One stratified 70/30 split
- Same executor and whitelist
- Same seed list in every cell

Temperature = 0.7, not 0

Run-to-run variance *is* the error term.
Greedy decoding would zero it — and take every standard error with it.

Four metrics, straight off the transcript

- **Gain over default** — did it improve anything?
- **Wasted trial ratio** — rejected + duplicate
- **Improvement rate** — trials beating the best so far
- **Cost to best** — calls to reach the peak

No judge model. No scoring library. No evaluation framework.

The model — and why the dataset is in it

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{j=2}^D \gamma_j D_j + \varepsilon$$

- ± 1 coding — orthogonal by construction
- γ_j absorbs task difficulty
- Without it, that variance inflates every standard error
- HC3 errors; Benjamini–Hochberg for multiplicity

What the loop actually did

Proposals attempted	5,140
Experiments executed	4,768
Proposals rejected	372
Duplicate proposals	5
Malformed replies	0

369 of 372 rejections were **whitelist violations**

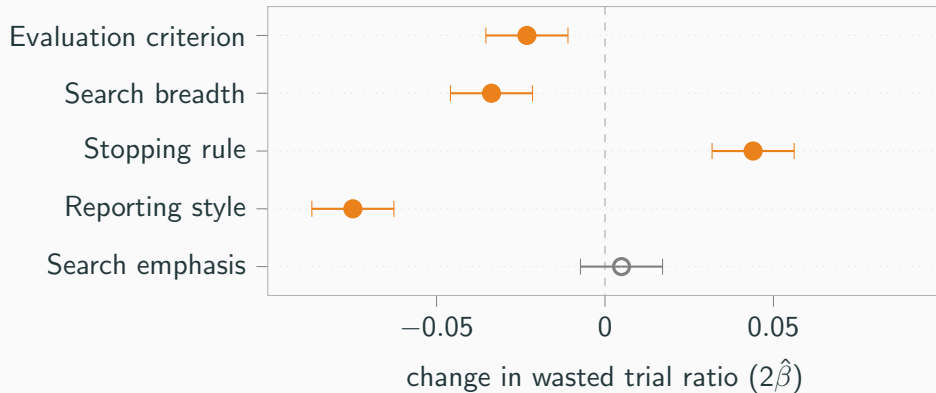
290: `min_samples_split`

71: `random_state`

Every one a legitimate scikit-learn argument the
whitelist does not admit

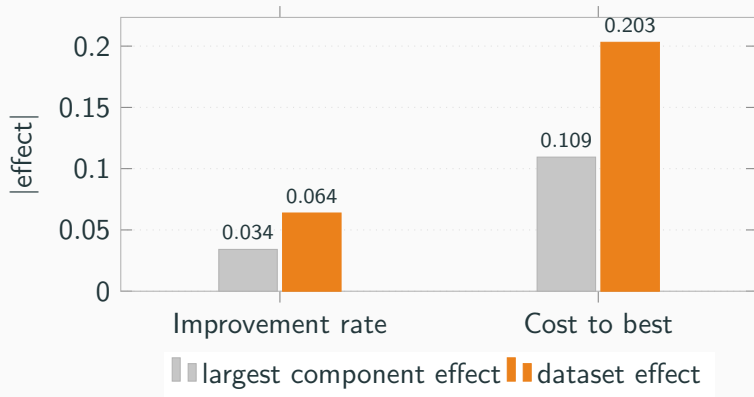
The loop is never shown the whitelist. It discovers the boundary by being rejected.

What actually moved



filled = survives Benjamini-Hochberg bars = 95% CI

The task matters more than the instructions



- Both dataset effects at $p < 10^{-5}$
- Adaptive stopping rule obeyed in only 20% of runs

RQ2: do the components interact?

Interaction	Est.	adj. p
$S \times O$	-0.031	0.000
$B \times O$	+0.018	0.009
$O \times E$	-0.016	0.019
$M \times B$	-0.010	0.171

3 of 40 survive. All on the wasted trial ratio,
all involving O . R^2 : 0.159 $\rightarrow$ 0.192.

Mean wasted trial ratio:

	terse	full sentences
fixed budget	0.068	0.024
adaptive budget	0.143	0.037

Additive prediction: 0.100. Observed: 0.037.

A generous budget is expensive *only* when the loop reports tersely.

What we proved

- Instructions govern **process**, not outcome
- Verbose reporting: wasted trials 10.6% $\rightarrow$ 3.1%
- Components close to additive (37 of 40 nulls)
- The method transfers to any prompt you can slice

$a_0 = 0.960$ on both tasks — the ceiling, not the loop, is the binding constraint

Questions

Backup — anticipated questions

Why not temperature 0?	It would zero the error term every coefficient is tested against.
$R^2 = 0.16$ — weak?	Expected for 5 factors over noisy agent behaviour. Hence $N = 1280$.
Only one agent model?	Stated limitation: one policy, not instructions in general.
One wording per level?	Stated limitation: effect confounded with phrasing. Top follow-up.
Why no compliance for M ?	Executor scores regardless, so M leaves no trace. O gets 205 vs 386 chars.
Only 40% beat baseline?	Ceiling effect: $a_0 = 0.960$; a further 39% return it exactly.

Backup — known defects of the treatment set

Levels length-matched?	No. Token counts differ up to $5\times$ (M : 10 vs 56). Length does not order the results: $r = -0.05$ with the wasted ratio across the 32 files.
Any cross-slot reference?	Yes, one. $S/1$ names “the evaluation metric (defined above)”, i.e. slot M , so $S = 1$ is underspecified when $M = 0$. $M \times S$ is null (adj. $p = 0.54$).
Seeds all as planned?	9 runs of 1,280 needed a replacement seed, in 2 cells. Every cell still holds exactly 20.
What does “refines” mean?	Same family + ≥ 1 hyperparameter value carried over. A weaker reading moves E compliance $0.69 \rightarrow 0.98$.
Meta-prompts available?	No. The ten paragraphs can be reused but not regenerated.

Backup — main effects

Metric	M	B	S	O	E	$\hat{\gamma}$	R^2
Gain over default	+0.001	-0.001	+0.001**	+0.000	+0.000	-0.001***	0.029
Wasted trial ratio	-0.023***	-0.034***	+0.044***	-0.075***	+0.005	-0.003	0.159
Improvement rate	+0.008	-0.026*	+0.034**	+0.006	+0.018	-0.064***	0.044
Cost to best	+0.06	-0.02	+0.05	-0.01	-0.11	-0.20***	0.025